

# 별무리

iOS 온디바이스 AI 개발 / 김민석

스마트폰에서 캐릭터와 대화하고 알람과 일정 같은 기기 기능을 연결하는 iOS 앱입니다. 소형 언어 모델을 기기 안에서 실행하며, 저장한 대화 내용을 검색해 이후 답변에 사용합니다.

**기간** 2026.05 - 현재  
**참여 과정** AI-SW 마에스트로 서울 제17기  
**담당** 온디바이스 AI 개발  
**진행 상태** TestFlight 사용자 테스트 중, 정식 출시 준비

## 전체 구조와 사용 기술

| 구성 요소    | 기술                      | 역할                                      |
|----------|-------------------------|-----------------------------------------|
| 클라이언트    | Unity                   | 사용자 입력과 세션 정보 전달, 생성 중인 답변 표시           |
| AI 요청 처리 | Swift                   | 대화와 도구 요청 분기, 기억 검색과 모델 입력 구성, 답변 조각 반환 |
| 언어 모델 추론 | Gemma 4 E2B / LiteRT-LM | 기기 CPU에서 대화 답변과 도구 호출 인자 생성             |
| 임베딩과 분류  | EmbeddingGemma / MLP    | 실행 의도 분류, 도구 선택과 기억 검색에 쓸 문장 벡터 생성      |
| 장기 기억    | SQLite                  | 저장 대상 발화와 벡터 보관, 현재 질문과 관련 있는 기억 검색     |

텍스트 생성과 임베딩은 기기에서 실행. 캐릭터 발화의 듣기 기능은 백엔드에서 음성 합성

## 담당 구현

- Unity 요청을 C API와 Swift 브릿지를 거쳐 LiteRT-LM 모델 호출로 연결
- EmbeddingGemma와 SQLite 기반 장기 기억의 저장과 검색 구현
- 실행 의도 분류와 도구 선택, KV 캐시 재사용과 파일 저장, 복원 구현
- 분류기 평가, 추론 결과 비교, 아이폰 자원 측정을 위한 평가 환경 개발

[소스 코드](#) / [기술 문서](#)

상세: [전체 요청 흐름](#)

# 일반 대화와 도구 실행 요청을 구분하도록 2단계 라우팅 구현

---

## 문제

- 여러 도구 설명을 한 번에 넣은 프롬프트에서 소형 모델이 일반 대화도 실행 요청으로 판단
- 임베딩 유사도로 바꾼 뒤에도 일반 대화 24건 중 11건이 호출 후보로 분류

## 원인

- 기능에 관한 질문, 과거 경험, 인용문도 도구 설명과 높은 임베딩 유사도를 가짐
- 도구를 언급한 문장과 실제 실행 요청을 유사도만으로 구분하기 어려움

## 수정

- EmbeddingGemma의 768차원 벡터를 입력받는 경량 MLP로 실행 의도부터 분류
- 분류를 통과한 발화만 도구별 대표 벡터와 비교해 도구 선택
- 선택한 도구 하나의 정의만 언어 모델에 전달해 JSON 인자 생성

## 검증

- 실제 실행 요청 168건과 일반 대화 24건으로 호출 후보 판정 비교
- 일반 대화 오인 11/24에서 1/24로 감소. 실제 실행 요청은 159/168 탐지
- 개발 중 확인한 평가셋의 재평가 결과이며, 새 평가셋에서의 성능은 미검증

## 적용 범위

- 한 요청에서 도구 하나만 선택. 여러 도구가 기준을 넘으면 요청을 나누도록 안내

상세: 실행 의도 분류와 도구 선택 / 호출 후보 판정 평가

# 앱 재실행 시 KV 캐시를 복원하도록 LiteRT-LM 수정

## 문제

- CachedSession으로 고정 프롬프트의 KV 캐시를 메모리에서 재사용
- 앱을 종료하면 캐시가 사라져, 재실행할 때 같은 프롬프트를 다시 계산

## 원인

- 사용하던 LiteRT-LM의 공개 C API에 추론 상태를 파일로 저장하고 복원하는 기능이 없었음

## 수정

- 엔진을 포크해 CPU 세션의 KV 텐서, 처리 위치, 토큰 ID, 샘플러 상태를 전송하는 C API 추가
- Swift에서 파일 저장과 새 세션 복원 처리. 복원 전 모델 정보와 체크섬 검사

## 검증

- Mac CPU의 별도 프로세스에서 복원한 뒤, 처음부터 계산한 결과와 출력 비교
- 다음 대화와 동적 입력 변경 조건에서 탐욕적 디코딩 출력 일치

## 남은 확인

- 아이폰에서 파일 저장과 복원 시간, 메모리 피크 측정

## 메모리 KV 캐시 재사용 측정

iPhone 17, 같은 5턴 대화를 조건별 3회 실행. 총 30개 요청 비교

| 측정 항목         | 표본 수    | 요청마다 새 세션 | 캐시 재사용  |
|---------------|---------|-----------|---------|
| 전체 평균 TTFT    | 조건별 15건 | 4.920초    | 1.274초  |
| 후속 요청 평균 TTFT | 조건별 12건 | 4.872초    | 0.446초  |
| 첫 요청 TTFT 중앙값 | 조건별 3건  | 4.570초    | 4.617초  |
| 후속 요청 평균 TPOT | 조건별 12건 | 39.29ms   | 38.39ms |

평균 TTFT 74.1% 감소, 후속 요청 평균 90.8% 감소. 답변 15쌍 모두 일치

2026.09.29 / iPhone 17, iOS 26.6.2 / Gemma 4 E2B, LiteRT-LM CPU / 컨텍스트 4,096, 최대 출력 128토큰 / temperature 0.7, top-k 40, top-p 1.0, thinking 끄 / 고정 지침과 사전 작성 대화, 기억 검색 끄

실행 중인 세션의 메모리 캐시 재사용을 비교한 결과. TTFT는 추론 호출부터 첫 출력 조각까지의 시간이며 기억 저장 판정 헤더 포함

상세: KV 캐시 구현과 검증 / 추론 엔진 포크 / 요청별 측정값